

Tecnológico de Monterrey
Inteligencia artificial avanzada para la ciencia de datos

Reporte

Predicción de jugadas de ajedrez a partir de la posición del tablero con redes neuronales convolucionales

---

Mauricio Olguín Sánchez
A01711522

Octubre 2026

Índice

1. Introducción **1**
- 1.1. Problema y objetivo . . . 1
- 1.2. Alcance . . . 1

2. El ajedrez en este proyecto **1**
- 2.1. Tablero y piezas . . . 1
- 2.2. Reglas especiales . . . 1
- 2.3. Cómo se escriben las posiciones y las partidas . . . 2
- 2.4. Jugadas, *plies* y fases de la partida . . . 2
- 2.5. Nivel de los jugadores y ritmo de juego . . . 2

3. Antecedentes **2**

4. Datos y proceso de ETL **3**
- 4.1. Sobre la base de datos seleccionada . . . 3
- 4.2. Proceso de ETL . . . 4
  - 4.2.1. *Extraction* . . . 4
  - 4.2.2. *Transforming* . . . 4
  - 4.2.3. *Loading* . . . 6
  - 4.2.4. Validación del proceso . . . 6

5. Análisis exploratorio de los datos **6**
- 5.1. Nivel de los jugadores . . . 6
- 5.2. Momento de la partida y fases . . . 7
- 5.3. Jugadas legales y la línea base del azar . . . 7
- 5.4. Las jugadas: clases, piezas y capturas . . . 7
- 5.5. Posiciones que se repiten entre partidas . . . 8
- 5.6. Conclusiones del análisis . . . 9

6. Metodología de evaluación **9**
- 6.1. Durante el entrenamiento . . . 9
- 6.2. Precisión *top-k* . . . 9
- 6.3. Líneas base . . . 10

7. Códigos **11**

Referencias **28**

1. Introducción

1.1. Problema y objetivo

El objetivo de este proyecto es construir, con técnicas de *deep learning*, un modelo que juegue ajedrez: que, dada una posición del tablero, elija una jugada.
Para lograrlo, el modelo aprende de partidas reales. Planteé el problema como una **clasificación multiclase supervisada**, con el mismo flujo de una red convolucional para Fashion MNIST, solo que con otro tipo de imagen y muchas más clases. El tablero se trata como una imagen pequeña: una cuadrícula de 8×8 casillas con varios canales, donde cada canal indica dónde están las piezas de un tipo y color. La red recibe esa imagen y produce, con una capa *softmax*, una probabilidad para cada jugada posible. La etiqueta de cada ejemplo es la jugada que de verdad se jugó en esa posición, y el modelo juega eligiendo la jugada más probable entre las legales.
Como aprende de partidas entre jugadores humanos de nivel alto, se espera que juegue parecido a ellos y no como un motor de ajedrez. Por eso, la primera forma de evaluarlo es medir qué tan seguido elige la jugada que hizo la persona.

1.2. Alcance

- **Núcleo:** una red que, a partir de una posición, elige la jugada. Se entrena con partidas de Lichess, también aprende a jugar la apertura y se evalúa con la precisión *top-k*.
- **Fuera de alcance:** aprendizaje por refuerzo, auto-juego y búsqueda MCTS. Quedan como trabajo futuro.

2. El ajedrez en este proyecto

Esta sección resume lo que hace falta saber de ajedrez para entender el resto del reporte: cómo es el tablero, qué reglas afectan la forma en que represento las posiciones y las jugadas, y cómo se escriben las partidas.

2.1. Tablero y piezas

El tablero tiene 8×8 casillas. Las columnas se nombran de la *a* a la *h* y las filas del 1 al 8, así que cada casilla tiene un nombre como e4. Las blancas empiezan en las filas 1 y 2 y las negras en las filas 7 y 8 (figura 1). Cada jugador tiene 16 piezas: un rey, una dama, dos torres, dos alfiles, dos caballos y ocho peones. La torre se mueve en línea recta, el alfil en diagonal, la dama en ambas direcciones, el rey una casilla en cualquier dirección y el caballo en forma de «L», saltando sobre las demás piezas. El peón avanza una casilla (dos en su primer movimiento) y captura en diagonal. Gana quien da **jaque mate**: atacar al rey rival de modo que no tenga forma de escapar.

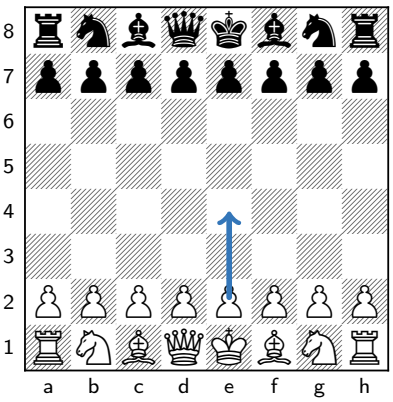


Figura 1: Posición inicial. La flecha muestra la jugada 1.e4.

Para el análisis uso el valor convencional de las piezas: peón 1, caballo 3, alfil 3, torre 5 y dama 9. El rey no tiene valor porque no se puede capturar.

2.2. Reglas especiales

Algunas reglas dependen de algo más que la posición de las piezas, y por eso la representación del tablero (sección 4) guarda información adicional:

- **Enroque:** el rey se mueve dos casillas hacia una torre y la torre salta al otro lado del rey. Solo se permite si ninguno de los dos se ha movido antes, por lo que hay que guardar los **derechos de enroque** de cada jugador.
- **Captura al paso:** si un peón avanza dos casillas y queda junto a un peón rival, este puede capturarlo como si hubiera avanzado una sola, pero únicamente en la jugada inmediata. Por eso guardo la columna en la que esa captura es posible.
- **Promoción:** el peón que llega a la última fila se convierte en otra pieza; casi siempre en dama.
- **Jaque y tablas:** un jugador no puede dejar a su rey en jaque. La partida termina en tablas por ahogado (no hay jugadas legales sin estar en jaque), por repetición, por la regla de los 50 movimientos, por material insuficiente o por acuerdo.

2.3. Cómo se escriben las posiciones y las partidas

- **SAN** (notación algebraica estándar): la forma habitual de escribir una jugada, con la letra de la pieza en inglés y la casilla de destino, por ejemplo `Nf3` (caballo a f3) o `e4` (peón a e4). Es la notación de los archivos de Lichess.
- **UCI**: la jugada como casilla de origen y de destino, por ejemplo `g1f3`. Es la base de mi etiqueta (sección 4).
- **FEN**: una línea de texto que describe una posición completa: piezas, a quién le toca, derechos de enroque, captura al paso y contadores. La posición inicial es: `rnbqkbnr/pppppppp/8/8/8/8/ PPPPPPPP/RNBQKBNR w KQkq - 0 1`.
- **PGN**: el formato de archivo de las partidas. Tiene encabezados (jugadores, Elo, tipo de partida, resultado) seguidos de las jugadas en SAN, y admite comentarios entre llaves.

2.4. Jugadas, *plies* y fases de la partida

En ajedrez, una «jugada» numerada incluye el movimiento de las blancas y la respuesta de las negras. En este reporte cuento ***plies*** (medias jugadas): cada movimiento de un solo jugador es un *ply*, así que la posición inicial es el *ply* 0.
Una partida suele dividirse en tres fases: la **apertura**, en la que se desarrollan las piezas y muchos jugadores repiten secuencias conocidas; el **medio juego**; y el **final**, cuando quedan pocas piezas. No existe una frontera única entre fases; la que uso la defino en el análisis exploratorio (sección 5).

2.5. Nivel de los jugadores y ritmo de juego

Lichess califica a cada jugador con un sistema de la familia Elo, y los archivos guardan esa calificación en los encabezados `WhiteElo` y `BlackElo`. En estos sistemas lo que importa es la diferencia entre dos jugadores: mientras mayor es, más probable es que gane el de mayor calificación. En este reporte llamo «Elo» a esa calificación.
Lichess también clasifica cada partida por su ritmo, según el tiempo disponible: Bullet (el más rápido), Blitz, Rápidas y Clásicas. El ritmo importa porque con muy poco tiempo las decisiones son más impulsivas.

3. Antecedentes

Antes de diseñar el modelo investigué otras implementaciones de este mismo problema. Al leer el artículo base [1] vi que su métrica, un R^2 sobre la diferencia de evaluación, no dice cuántas veces el modelo elige la jugada correcta. De ahí surgió investigar también cómo evaluar un modelo que aprende de jugadas humanas. La tabla 1 resume los trabajos que influyeron en mis decisiones. Sus cifras no son comparables entre sí, porque cada uno usa datos, objetivos y criterios de acierto diferentes.

Trabajo	Resultado principal	Diferencia con mi trabajo
Arsalan y Soeparno [1]	CNN en Keras sobre 3,862 partidas de grandes maestros de Lichess. Regresión de la diferencia de evaluación; $R^2 = 0{,}175$ en el mejor modelo y 4 de 28 problemas tácticos resueltos.	Es el artículo base. Codifica cada pieza con un entero, lo que impone un orden sin sentido entre piezas; yo uso un canal por pieza. No tiene conjunto de prueba independiente y reporta el mejor de 400 modelos; yo reporto la media entre semillas. Ellos hacen regresión; yo clasifico la jugada.
McIlroy-Young et al. (Maia) [5]	Red residual entrenada con unos 12 millones de partidas de Lichess por modelo. Acierta entre 46 % y más de 52 % de las jugadas humanas.	De su protocolo tomo la evaluación solo entre jugadas legales, la división por partida y la exclusión de las partidas Bullet.
Soldera y Pedrini [6]	CNN con entrada $8 \times 8 \times 12$ y salida separada en pieza y destino; 68.5 % con un híbrido con Stockfish.	Usa la misma idea de un canal por pieza. Su criterio de acierto es más laxo (cualquier jugada humana registrada para la posición) y solo usan una apertura.
Bertram et al. [2]	Red para una variante de ajedrez con información imperfecta; tablero como 12 planos de 8×8. Usar casi solo convoluciones rindió mejor que añadir capas densas, y preentrenar con autoencoders no mejoró el resultado.	Apoya la representación con un canal por pieza. Su resultado con autoencoders es un riesgo conocido si uso un autoencoder como base del modelo.

Cuadro 1: Trabajos que influyeron en las decisiones del proyecto. Las métricas pertenecen a problemas y protocolos distintos.

Estos trabajos muestran que predecir la jugada exacta es difícil: Maia, con una red grande y millones de partidas, acierta alrededor de la mitad. Como sus cifras usan otros datos y criterios, no compararé mi modelo directamente con ellas, sino con **líneas base calculadas con mis propios datos** (sección 6.3). Además, Maia descarta las primeras 10 jugadas de cada partida; yo no, porque quiero que el modelo también sepa jugar la apertura. Por eso la incluyo y reporto el resultado por fase de la partida.

4. Datos y proceso de ETL

4.1. Sobre la base de datos seleccionada

Utilicé la *Lichess Open Database* [4], que publica cada mes todas las partidas jugadas en Lichess con licencia CC0. Cada archivo está en formato PGN comprimido con Zstandard (`.pgn.zst`). Los meses recientes pesan alrededor de 30 GB comprimidos cada uno, mientras que los primeros meses de la base son mucho más pequeños.

Primero construí y probé el *pipeline* con el archivo de marzo de 2013 (`lichess_db_standard_rated_2013-03.pgn.zst`), que pesa 22.5 MB comprimido y contiene 158,635 partidas con calificación (*rated*). Ese mes tiene una limitación clara: casi no hay jugadores de nivel alto. Solo en 13 partidas que no son Bullet ambos jugadores tienen un Elo de 2,200 o más.

Por eso el conjunto definitivo sale del mes completo más reciente al momento de procesarlo, septiembre de 2026 (`lichess_db_standard_rated_2026-09.pgn.zst`): pesa 29.2 GB comprimido y contiene 89,616,462 partidas. La tabla 2 compara los dos meses. Cada partida trae encabezados con el tipo de evento, el Elo de ambos jugadores, el resultado, el control de tiempo y la forma en

que terminó, seguidos de las jugadas en notación algebraica. En los meses recientes los encabezados también incluyen el título del jugador, como GM o BOT (las cuentas de programas de ajedrez), y las jugadas vienen acompañadas de comentarios con el tiempo que le quedaba a cada jugador.

Característica	2013-03	2026-09
Tamaño comprimido	22.5 MB	29.2 GB
Partidas	158,635	89,616,462
Bullet (con UltraBullet)	43,548	32,517,004
Blitz	60,655	41,228,526
Rápidas	–	15,224,219
Clásicas	54,155	555,136
Por correspondencia	277	91,577
En torneos	2,030	7,353,507
Con alguna cuenta BOT	0	434,571
Victorias de las blancas	80,486	44,474,193
Victorias de las negras	72,796	41,504,000
Tablas	5,353	3,417,815
Terminadas normalmente	110,271	60,590,231
Terminadas por tiempo	48,364	28,564,435
Elo de las blancas (mín.)	802	400
Elo de las blancas (media)	1,607	1,644
Elo de las blancas (máx.)	2,412	3,602

Cuadro 2: Características de los dos meses, calculadas leyendo los encabezados de todas las partidas. En 2013 Lichess todavía no tenía la categoría de partidas rápidas.

El Elo medio de los dos meses es parecido, pero en 2026 el rango es mucho más amplio: hay más de un millón y medio de partidas que no son Bullet entre jugadores humanos con un Elo de 2,200 o más. De esas partidas sale el conjunto que uso.

4.2. Proceso de ETL

Para leer las partidas y generar las jugadas legales utilicé la biblioteca `chess` [3].

4.2.1. *Extraction*

El archivo se lee en *streaming*: se descomprime por bloques y se procesa una partida a la vez. Así nunca se guarda el PGN descomprimido en disco, que en un mes reciente ocuparía cientos de gigabytes, ni se carga completo en memoria.

Reconstruir el tablero jugada por jugada es el paso más lento, y en un mes reciente la gran mayoría de las partidas se descarta. Por eso primero leo solo los encabezados de cada partida y aplico con ellos los filtros. Únicamente analizo las jugadas de las partidas que los pasan. Con esto, el mes completo se procesa en unos 25 minutos.

4.2.2. *Transforming*

Filtros Apliqué los filtros en este orden (tabla 3):

- **Ritmo:** descarté las partidas Bullet y UltraBullet, porque con tan poco tiempo las jugadas son más impulsivas y menos representativas.
- **Cuentas BOT:** descarté las partidas en las que juega alguna cuenta BOT. Son programas de ajedrez, y quiero que el modelo aprenda de jugadas humanas. Este filtro lo agregué a partir del análisis exploratorio (sección 5).
- **Nivel:** conservé solo las partidas en las que ambos jugadores tienen un Elo de 2,200 o más, para que el modelo aprenda de jugadores de nivel alto. Este filtro deja fuera a la gran mayoría de las partidas del mes.
- **Muestra aleatoria:** las partidas que pasan los filtros anteriores son 1,507,816. Con 12 posiciones cada una serían unos 18 millones de posiciones, más de lo que mi computadora puede procesar con holgura y demasiado para entrenar en un tiempo razonable sin GPU. Por eso tomé al azar una de cada cuatro, con una semilla fija. Muestrear todo el mes, en lugar de quedarme con sus primeros días, evita que el conjunto dependa de unas fechas en particular.

Ninguna partida venía sin Elo ni con errores de lectura, y 468 de las elegidas no tenían jugadas.

Paso	Partidas
Partidas leídas	89,616,462
Bullet o UltraBullet	32,517,004
Con alguna cuenta BOT	325,060
Algún jugador con Elo menor a 2,200	55,266,582
Fuera de la muestra aleatoria	1,130,890
Sin jugadas	468
Partidas utilizadas	**376,458**

Cuadro 3: Partidas descartadas en cada filtro del mes de septiembre de 2026, en el orden en que se aplican. Las partidas BOT que también son Bullet se cuentan como Bullet.

El tablero visto desde quien mueve Para que la red no tenga que aprender dos veces lo mismo (una vez para las blancas y otra para las negras), represento siempre la posición **desde el punto de vista del jugador que mueve**. Cuando mueven las negras, volteo el tablero verticalmente e intercambio los colores, de modo que el jugador que mueve siempre aparece abajo.
Este punto se relaciona con el aumento de datos, cuyas transformaciones deben respetar la lógica del problema. En ajedrez, voltear el tablero de izquierda a derecha **no** es válido en general, porque el rey y la dama no son simétricos y el enroque cambiaría de sentido. La única simetría que se puede usar sin cuidado especial es la de voltear verticalmente e intercambiar los colores.

Representación del tablero La red recibe cada posición como una «imagen» de 8×8 con 17 canales binarios (tabla 4): uno por cada tipo de pieza y color, más los derechos de enroque y la captura al paso.

Canales	Contenido
0–5	Peones, caballos, alfiles, torres, dama y rey del jugador que mueve
6–11	Las mismas piezas del rival
12–15	Derechos de enroque (corto y largo de cada jugador), canal completo en 1 o en 0
16	Casilla de captura al paso, si existe

Cuadro 4: Canales de la representación *one-hot* del tablero.

Para compararme con el artículo base también genero su versión ordinal: un solo canal donde cada pieza vale de 1 a 6 y el signo indica si es del jugador que mueve o del rival.

Representación de la jugada La etiqueta es la jugada que se jugó, codificada como un número entre 0 y 4,095:

$$\text{clase} = 64 \times \text{origen} + \text{destino},$$

donde origen y destino son las casillas numeradas de 0 (a1) a 63 (h8), recorriendo cada fila de izquierda a derecha, ya en la orientación de quien mueve. Por ejemplo, 1.e4 (de e2, casilla 12, a e4, casilla 28) es la clase $64 \times 12 + 28 = 796$. Gracias a la orientación, 1.e4 de las blancas y 1. . . e5 de las negras tienen la misma clase. El enroque queda representado como un movimiento del rey de dos casillas. Las promociones se representan como promoción a dama; las 105 posiciones en las que se jugó una promoción a otra pieza se omitieron.

Jugadas legales Para cada posición guardo también la lista de sus jugadas legales y el valor de la pieza capturada por cada una (peón 1, caballo y alfil 3, torre 5, dama 9). Esto permite descartar las jugadas ilegales al evaluar el modelo y calcular las líneas base sin tener que reconstruir los tableros.

Muestreo de posiciones Las posiciones consecutivas de una misma partida se parecen mucho entre sí, así que no tiene sentido usar todas. De cada partida tomé al azar

12 posiciones (o todas, si la partida es más corta), con una semilla fija para que el resultado sea reproducible. Después de omitir las subpromociones, el resultado fue de **4,505,669 posiciones**.

4.2.3. *Loading*

El conjunto procesado se guarda en disco para no repetir la transformación cada vez que se entrena.
La división en entrenamiento, validación y prueba se hace **por partida y no por posición**. Si se dividiera por posición, posiciones casi idénticas de la misma partida quedarían en entrenamiento y en prueba, y las métricas saldrían infladas por una fuga de datos. Mezclé las partidas con la semilla 42 y asigné 80 %, 10 % y 10 % de ellas a cada conjunto (tabla 5). Ninguna partida aparece en más de un conjunto.

Conjunto	Partidas	Posiciones
Train	301,166	3,604,441
Validation	37,645	450,591
Test	37,647	450,637

Cuadro 5: División por partida con semilla 42.

El conjunto de *test* no interviene en ninguna decisión: la elección de arquitecturas e hiperparámetros se hará con *validation*, y *test* se usará una sola vez al final.

4.2.4. Validación del proceso

Como un error en la codificación invalidaría todos los resultados, escribí pruebas automáticas antes de procesar los datos. Así, cualquier cambio posterior que rompa la codificación se detecta de inmediato. Entre otras cosas, comprueban que la posición inicial tenga 20 jugadas legales, que cada jugada se pueda codificar y decodificar sin pérdida (incluidos el enroque, la captura al paso y la promoción), que una posición con negras y su versión espejada tengan la misma codificación, que las etiquetas coincidan con las jugadas reales de partidas de ejemplo, que cada filtro descarte las partidas correctas y que la división no repita partidas entre conjuntos.
Cuando agregué la lectura previa de los encabezados, comprobé que, sin los filtros nuevos, el proceso producía exactamente el mismo conjunto que antes con el mes de 2013.

5. Análisis exploratorio de los datos

Antes de entrenar revisé el conjunto procesado (4,505,669 posiciones) para conocer a los jugadores, el momento de la partida en que ocurre cada posición, las jugadas que se eligen y qué tanto se repiten las posiciones entre partidas.

5.1. Nivel de los jugadores

El Elo del jugador que mueve va de 2,213 a 2,585 en el 90 % central de las posiciones, con mediana de 2,327 y máximo de 3,010 (figura 2). El 29.0 % de las posiciones corresponde a jugadores con 2,400 o más y el 1.2 %, a jugadores con 2,700 o más. En el mes de 2013, en cambio, la mediana era de 1,574 y el máximo de 2,359.
En la primera versión de este histograma apareció un segundo pico alrededor de 3,000. Al revisar los encabezados encontré que eran cuentas BOT. En las primeras 28 horas del mes, el 5.7 % de las partidas que no son Bullet entre jugadores de 2,200 o más tenía al menos una cuenta BOT, casi siempre una contra otra, con un Elo mediano de 2,718. Como son jugadas de programas y no de personas, excluí esas partidas en el ETL.

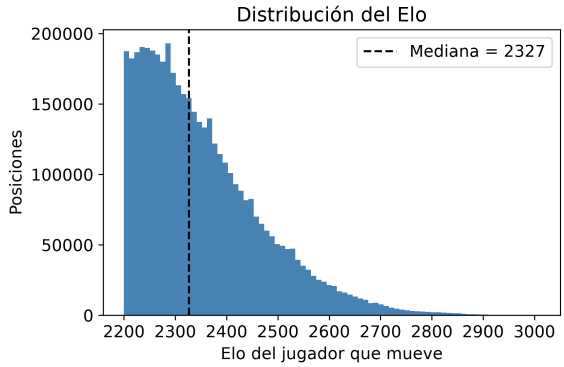


Figura 2: Elo del jugador que mueve, ya sin las partidas de cuentas BOT.

5.2. Momento de la partida y fases

Las posiciones se concentran en los primeros *plies* y su frecuencia baja conforme avanza la partida, porque muchas partidas terminan antes (figura 3); el *ply* promedio es 39.1. El 82.9 % de las posiciones ocurre después del *ply* 10.

Para analizar los resultados por fase definí tres grupos. El **final** son las posiciones con 12 piezas o menos, contando a los reyes; la **apertura**, el resto de las posiciones antes del *ply* 20 (diez jugadas de cada bando); y el **medio juego**, todo lo demás. Con esta definición, el 30.8 % de las posiciones es de apertura, el 60.1 % de medio juego y el 9.2 % de final (figura 4). Como el final tiene pocas posiciones, sus resultados tendrán más incertidumbre.

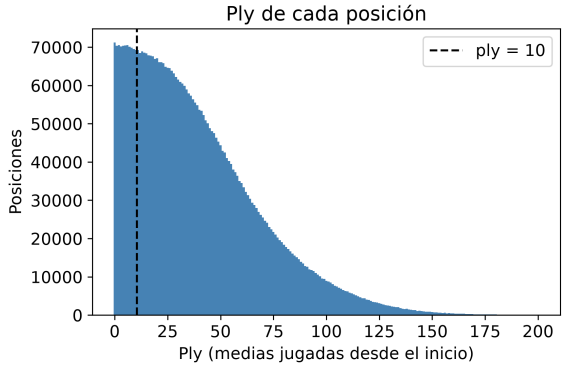


Figura 3: *Ply* en el que ocurre cada posición muestreada.

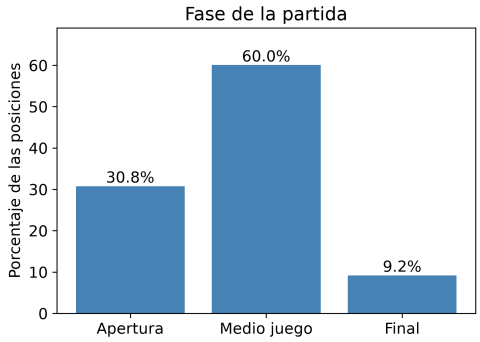


Figura 4: Posiciones por fase de la partida.

5.3. Jugadas legales y la línea base del azar

Cada posición tiene en promedio **31.0 jugadas legales**, con un mínimo de 1 y un máximo de 100 (figura 5). El pico en 20 corresponde a la posición inicial y a otras posiciones de apertura muy parecidas. El 0.6 % de las posiciones tiene una sola jugada legal y el 4.0 %, cuatro o menos.

Si se elige una jugada legal al azar, se acierta el **5.4 %** de las veces en promedio. Es más que $1/31{,}0 \approx 3{,}2\,\%$ porque en las posiciones con pocas opciones, como las de jaque, el azar acierta mucho más. Esta es la primera línea base que el modelo debe superar.

5.4. Las jugadas: clases, piezas y capturas

Solo aparecen 1,792 de las 4,096 clases posibles, porque muchas combinaciones de origen y destino no corresponden al movimiento de ninguna pieza. Además, las clases que sí aparecen están muy desbalanceadas. La figura 6 ordena las jugadas de la más a la menos común y muestra qué porcentaje del total se acumula: las 74 jugadas más comunes ya son la mitad de todas las jugadas del conjunto, 348 son el 80 % y 989 el 95 %. Las otras 803 juntas apenas suman el 5 % restante. Esto significa que el modelo verá muchísimos ejemplos de unas pocas jugadas y muy pocos del resto.

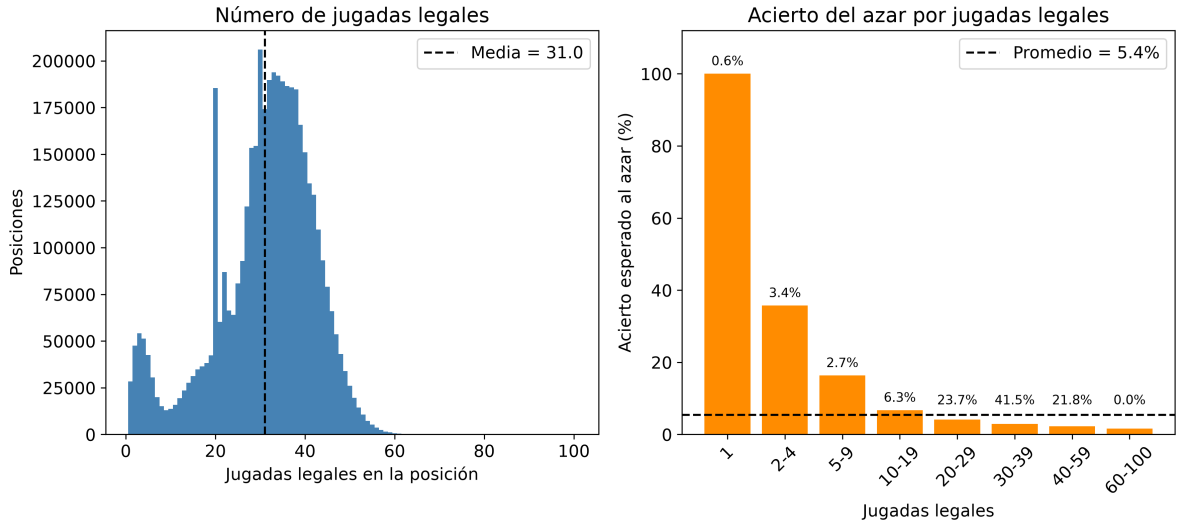


Figura 5: Izquierda: número de jugadas legales por posición. Derecha: acierto esperado al elegir una jugada legal al azar, por intervalo; encima de cada barra, el porcentaje de posiciones del intervalo.

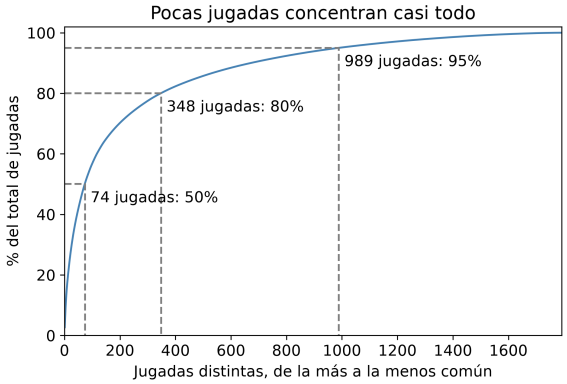


Figura 6: Porcentaje acumulado de jugadas al sumar las clases de la más a la menos común. Las líneas punteadas marcan cuántas clases hacen falta para cubrir el 50 %, el 80 % y el 95 % de las jugadas.

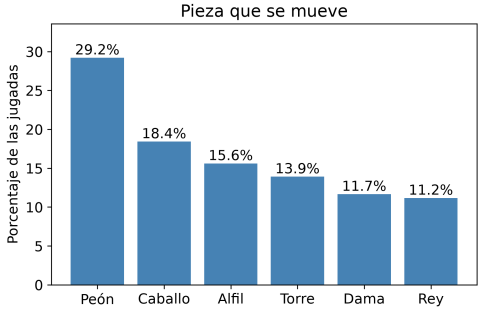


Figura 7: Tipo de pieza que se mueve en cada jugada.

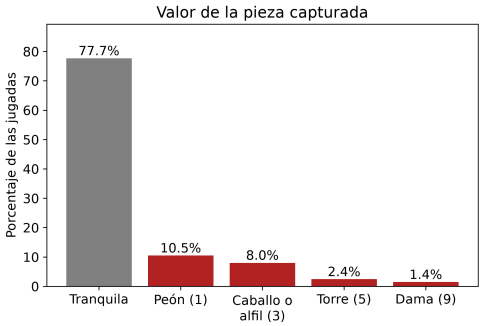


Figura 8: Jugadas tranquilas y capturas, según el valor de la pieza capturada.

La pieza que más se mueve es el peón (29.2 %), seguida del caballo (18.4 %) y el alfil (15.6 %); la torre, la dama y el rey se mueven entre el 11.2 % y el 13.9 % de las veces (figura 7).

El 22.3 % de las jugadas son capturas (figura 8). En el 75.7 % de las posiciones hay al menos una captura disponible, pero solo en el 25.0 % de ellas se juega la captura de mayor valor. Por eso espero que la línea base de «capturar la pieza más valiosa» sea débil: la mayoría de las veces los jugadores no capturan aunque puedan.

5.5. Posiciones que se repiten entre partidas

Como incluí las aperturas, quise saber cuántas posiciones aparecen idénticas (mismas

piezas, derechos de enroque y captura al paso) en dos o más partidas distintas. Se repiten porque muchos jugadores juegan las mismas aperturas, y eso significa que una parte de lo que aprenda el modelo en esas posiciones puede ser memoria.

El 21.6 % de las posiciones aparece en al menos dos partidas. Casi todas son de apertura: ahí se repite el 68.0 % de las posiciones, mientras que en el resto de la partida solo el 1.0 %. Por *ply*, la posición inicial se repite siempre, en el *ply* 4 el 98.6 %, en el *ply* 10 el 72.1 %, en el *ply* 16 el 28.7 % y desde el *ply* 26 menos del 2 % (figura 9). Las partidas de nivel alto siguen caminos conocidos durante más jugadas, probablemente porque estos jugadores conocen más teoría de aperturas: en el mes de 2013, en el *ply* 10 solo se repetía el 29.8 % de las posiciones.

En esas posiciones repetidas, la jugada más frecuente se elige el 62.1 % de las veces. Es decir, en una misma posición distintos jugadores eligen jugadas distintas, y ningún modelo que solo vea el tablero puede acertar todas: en estas posiciones, el acierto *top-1* tiene un techo cercano al 62 %. En la posición inicial el techo es todavía más bajo, del 48.1 %, porque los jugadores reparten su primera jugada entre 1.e4 (48.1 %), 1.d4 (32.2 %), 1. Nf3 (8.4 %) y 1.c4 (5.9 %), entre otras.

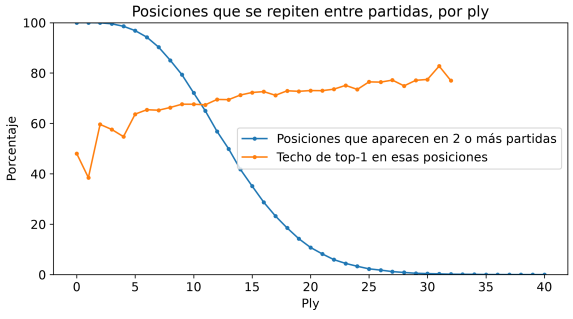


Figura 9: Por *ply*: porcentaje de posiciones que aparecen en dos o más partidas y techo del acierto *top-1* en esas posiciones. Solo se grafican los *plies* con al menos 100 posiciones.

5.6. Conclusiones del análisis

- Usé solo partidas entre jugadores humanos con un Elo de 2,200 o más; las cuentas BOT quedaron fuera.
- Las fases quedan definidas por *ply* (apertura antes del 20) y por número de piezas (final con 12 o menos), y reportaré los resultados de cada una por separado.
- El modelo tiene que superar al azar (5.4 %) en todo el conjunto.
- Más de dos tercios de las posiciones de apertura se repiten entre partidas, así que en la apertura también lo compararé con el «libro» (sección 6.3), que solo usa memoria.
- En las posiciones repetidas, el acierto *top-1* no puede pasar de alrededor del 62 %, porque los jugadores no coinciden entre sí.
- La cola larga de clases y el bajo porcentaje de capturas de mayor valor indican que el modelo tendrá que aprender jugadas poco frecuentes y que una regla sencilla no bastará.

6. Metodología de evaluación

6.1. Durante el entrenamiento

La red se entrena con la entropía cruzada categórica (*sparse categorical crossentropy*), la pérdida para clasificación cuando las etiquetas son enteros. En cada época registro la pérdida y la precisión en entrenamiento y validación. Con *ModelCheckpoint* guardo el modelo de menor pérdida en validación, de modo que el número de épocas no tiene que adivinarse de antemano.

6.2. Precisión *top-k*

La métrica principal es la precisión *top-1*: el porcentaje de posiciones en las que la ju-

gada más probable del modelo es la que se jugó. También reporto *top-3* y *top-5* (si la jugada real está entre las tres o cinco más probables). Antes de calcularlas descarto las jugadas ilegales, porque en una partida nunca podrían elegirse. Como la red se entrena sin esa restricción, también mido qué porcentaje de sus primeras opciones es ilegal, para saber si aprendió las reglas.
La métrica principal se calcula sobre **todas** las posiciones, incluidas las de apertura. Además la desgloso por fase (apertura, medio juego y final). Solo como referencia, para compararme con el protocolo de Maia, también la calculo sin los primeros 10 *plies*.

6.3. Líneas base

Todas las métricas se comparan con cuatro líneas base calculadas con los mismos datos:

1. una jugada legal al azar (5.4 % de acierto esperado);
2. la jugada legal más frecuente en el conjunto de entrenamiento;
3. la captura de mayor valor, si existe;
4. un «libro»: si la posición exacta aparece en entrenamiento, la jugada que más se jugó en ella. Esta línea base mide cuánto se puede acertar solo con memoria, sobre todo en la apertura, donde las mismas posiciones se repiten en muchas partidas.

7. Códigos

Todos los códigos están en https://github.com/TheSimpleMau/Portafolio-Implementacion-Deeplearning. Aquí se incluyen los módulos principales hasta el momento. Codificación del tablero y de las jugadas.

```python
import chess
import numpy as np

N_CLASSES = 64 * 64
N_PIECE_PLANES = 12
N_PLANES = 17

PIECE_VALUES = {
    chess.PAWN: 1,
    chess.KNIGHT: 3,
    chess.BISHOP: 3,
    chess.ROOK: 5,
    chess.QUEEN: 9,
    chess.KING: 0,
}


def orient(board: chess.Board):
    if board.turn == chess.BLACK:
        return board.mirror(), True
    return board, False


def encode_position(board: chess.Board):
    board, _ = orient(board)

    squares = np.zeros(64, dtype=np.int8)
    for square, piece in board.piece_map().items():
        squares[square] = piece.piece_type + (0 if piece.color == chess.WHITE else 6)

    castling = (
        int(board.has_kingside_castling_rights(chess.WHITE))
        | int(board.has_queenside_castling_rights(chess.WHITE)) << 1
        | int(board.has_kingside_castling_rights(chess.BLACK)) << 2
        | int(board.has_queenside_castling_rights(chess.BLACK)) << 3
    )
    ep = chess.square_file(board.ep_square) if board.has_legal_en_passant() else -1
    return squares, castling, ep


def encode_move(move: chess.Move, flipped: bool):
```

```
42      if move.promotion not in (None, chess.QUEEN):
43          return None
44      from_square, to_square = move.from_square, move.to_square
45      if flipped:
46          from_square = chess.square_mirror(from_square)
47          to_square = chess.square_mirror(to_square)
48      return from_square * 64 + to_square
49
50
51  def decode_move(index: int, board: chess.Board):
52      from_square, to_square = divmod(int(index), 64)
53      if board.turn == chess.BLACK:
54          from_square = chess.square_mirror(from_square)
55          to_square = chess.square_mirror(to_square)
56
57      promotion = None
58      if (
59          board.piece_type_at(from_square) == chess.PAWN
60          and chess.square_rank(to_square) in (0, 7)
61      ):
62          promotion = chess.QUEEN
63      return chess.Move(from_square, to_square, promotion)
64
65
66  def _capture_value(board: chess.Board, move: chess.Move):
67      if board.is_en_passant(move):
68          return PIECE_VALUES[chess.PAWN]
69      captured = board.piece_type_at(move.to_square)
70      return PIECE_VALUES[captured] if captured else 0
71
72
73  def legal_moves_info(board: chess.Board):
74      flipped = board.turn == chess.BLACK
75      captures = {}
76      for move in board.legal_moves:
77          index = encode_move(move, flipped)
78          if index is None:
79              continue
80          captures[index] = _capture_value(board, move)
81
82      indices = np.array(sorted(captures), dtype=np.int16)
83      values = np.array([captures[i] for i in indices.tolist()],
            dtype=np.int8)
84      return indices, values
85
86
87  def expand_planes(squares: np.ndarray, castling: np.ndarray, ep:
        np.ndarray):
88      n = len(squares)
89      board = squares.reshape(n, 8, 8)
90      planes = np.zeros((n, 8, 8, N_PLANES), dtype=np.uint8)
```

```
91
92      for code in range(1, N_PIECE_PLANES + 1):
93          planes[..., code - 1] = board == code
94
95      castling = np.asarray(castling)
96      for bit in range(4):
97          planes[..., N_PIECE_PLANES + bit] = ((castling >> bit) &
               1)[:, None, None]
98
99      ep = np.asarray(ep)
100     has_ep = np.flatnonzero(ep >= 0)
101     planes[has_ep, 5, ep[has_ep], N_PLANES - 1] = 1
102     return planes
103
104
105 def expand_ordinal(squares: np.ndarray):
106     n = len(squares)
107     codes = squares.astype(np.int8)
108     ordinal = np.where(codes > 6, -(codes - 6), codes).astype(np.
           int8)
109     return ordinal.reshape(n, 8, 8, 1)
```

Proceso de ETL.

```python
import io
import json
import os
import time

import chess
import chess.pgn
import numpy as np
import zstandard

from preprocessing import encode_move, encode_position, legal_moves_info

# 2013-03 solo sirve para depurar; el conjunto principal es el mes más reciente.
MONTH = "2026-09"
RAW_DATA_PATH = f"data/raw/lichess_db_standard_rated_{MONTH}.pgn.zst"
PROCESSED_PATH = f"data/processed/positions_{MONTH}.npz"
SUMMARY_PATH = f"data/processed/positions_{MONTH}_resumen.json"

POSITIONS_PER_GAME = 12
# También excluye UltraBullet.
EXCLUDED_EVENTS = ("Bullet",)
# Cuentas de motores: sus jugadas no son humanas.
EXCLUDED_TITLES = ("BOT",)
# Elo mínimo de ambos jugadores.
MIN_ELO = 2200
# Fracción de las partidas que pasan los filtros que se analizan.
GAME_SAMPLE_RATE = 0.25
# Pico en 1500: probablemente jugadores nuevos con la calificación inicial.
EXCLUDED_MOVER_ELOS = (1500,)

TRAIN_FRACTION = 0.8
VAL_FRACTION = 0.1
TEST_FRACTION = 0.1
RANDOM_STATE = 42

PROGRESS_EVERY = 10_000


def read_games(path):
    """Separa el PGN en partidas sin analizar las jugadas: (cabeceras, texto)."""
    with open(path, "rb") as raw:
        reader = zstandard.ZstdDecompressor().stream_reader(raw)
        headers, lines = {}, []
        for line in io.TextIOWrapper(reader, encoding="utf-8"):
            if line.startswith("[Event ") and lines:
```

```
                yield headers, "".join(lines)
                headers, lines = {}, []
            if line.startswith("["):
                key, _, value = line.strip()[1:-1].partition(" ")
                headers[key] = value.strip('"')
            lines.append(line)
        if lines:
            yield headers, "".join(lines)


def _parse_elo(value):
    return int(value) if value and value.isdigit() else None


def _header_discard_reason(headers, min_elo):
    if any(event in headers.get("Event", "") for event in
       EXCLUDED_EVENTS):
        return "bullet"
    if headers.get("WhiteTitle") in EXCLUDED_TITLES or headers.get
       ("BlackTitle") in EXCLUDED_TITLES:
        return "bot"
    elos = (_parse_elo(headers.get("WhiteElo")), _parse_elo(
       headers.get("BlackElo")))
    if None in elos:
        return "sin_elo"
    if min(elos) < min_elo:
        return "elo_bajo"
    return None


def _game_discard_reason(game):
    if game.errors:
        return "error_pgn"
    if game.next() is None:
        return "sin_jugadas"
    return None


def transform_data(games, positions_per_game=POSITIONS_PER_GAME,
   min_elo=MIN_ELO,
                   game_sample_rate=GAME_SAMPLE_RATE, random_state
                      =RANDOM_STATE,
                   max_games=None):
    rng = np.random.default_rng(random_state)
    counters = {
        "partidas_leidas": 0,
        "partidas_usadas": 0,
        "bullet": 0,
        "bot": 0,
        "sin_elo": 0,
        "elo_bajo": 0,
```

```
        "fuera_de_muestra": 0,
        "error_pgn": 0,
        "sin_jugadas": 0,
        "subpromociones": 0,
        "posiciones_elo_excluido": 0,
    }
    columns = {name: [] for name in ("squares", "castling", "ep",
       "move", "game_id", "ply", "elo")}
    legal_moves, legal_capture, legal_counts = [], [], []
    started = time.perf_counter()

    for headers, pgn in games:
        if max_games is not None and counters["partidas_leidas"]
           >= max_games:
            break
        counters["partidas_leidas"] += 1
        if counters["partidas_leidas"] % PROGRESS_EVERY == 0:
            print(
                f"{counters['partidas_leidas']:,} partidas leídas,
                    "
                f"{len(columns['move']):,} posiciones "
                f"({time.perf_counter() - started:.0f} s)"
            )

        reason = _header_discard_reason(headers, min_elo)
        if reason:
            counters[reason] += 1
            continue
        # Solo se sortea con muestreo para que con 1.0 los
           resultados no cambien.
        if game_sample_rate < 1 and rng.random() >=
           game_sample_rate:
            counters["fuera_de_muestra"] += 1
            continue
        game = chess.pgn.read_game(io.StringIO(pgn))
        reason = _game_discard_reason(game)
        if reason:
            counters[reason] += 1
            continue

        game_id = counters["partidas_usadas"]
        counters["partidas_usadas"] += 1
        elos = {
            chess.WHITE: _parse_elo(game.headers["WhiteElo"]),
            chess.BLACK: _parse_elo(game.headers["BlackElo"]),
        }
        moves = list(game.mainline_moves())
        n_samples = min(positions_per_game, len(moves))
        sampled = set(rng.choice(len(moves), size=n_samples,
           replace=False).tolist())
        last_ply = max(sampled)
```

```
137
138         board = game.board()
139         for ply, move in enumerate(moves):
140             if ply in sampled:
141                 index = encode_move(move, board.turn == chess.
                        BLACK)
142                 if elos[board.turn] in EXCLUDED_MOVER_ELOS:
143                     counters["posiciones_elo_excluido"] += 1
144                 elif index is None:
145                     counters["subpromociones"] += 1
146                 else:
147                     squares, castling, ep = encode_position(board)
148                     indices, captures = legal_moves_info(board)
149                     columns["squares"].append(squares)
150                     columns["castling"].append(castling)
151                     columns["ep"].append(ep)
152                     columns["move"].append(index)
153                     columns["game_id"].append(game_id)
154                     columns["ply"].append(ply)
155                     columns["elo"].append(elos[board.turn])
156                     legal_moves.append(indices)
157                     legal_capture.append(captures)
158                     legal_counts.append(len(indices))
159             if ply == last_ply:
160                 break
161             board.push(move)
162
163     data = {
164         "squares": np.stack(columns["squares"]).astype(np.int8),
165         "castling": np.array(columns["castling"], dtype=np.uint8),
166         "ep": np.array(columns["ep"], dtype=np.int8),
167         "move": np.array(columns["move"], dtype=np.int16),
168         "game_id": np.array(columns["game_id"], dtype=np.int32),
169         "ply": np.array(columns["ply"], dtype=np.int16),
170         "elo": np.array(columns["elo"], dtype=np.int16),
171         "legal_offsets": np.concatenate([[0], np.cumsum(
                legal_counts)]).astype(np.int64),
172         "legal_moves": np.concatenate(legal_moves).astype(np.int16
                ),
173         "legal_capture": np.concatenate(legal_capture).astype(np.
                int8),
174     }
175     counters["posiciones"] = len(data["move"])
176     counters["duracion_segundos"] = round(time.perf_counter() -
            started, 1)
177     return data, counters
178
179
180 def save_data(data, counters, path=PROCESSED_PATH, summary_path=
        SUMMARY_PATH):
181     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
```

```
182     np.savez(path, **data)
183     with open(summary_path, "w", encoding="utf-8") as file:
184         json.dump(counters, file, indent=2, ensure_ascii=False)
185
186
187 def load_data(path=PROCESSED_PATH):
188     with np.load(path) as file:
189         return {key: file[key] for key in file.files}
190
191
192 def split_data(game_ids, train_fraction=TRAIN_FRACTION,
    val_fraction=VAL_FRACTION,
193                test_fraction=TEST_FRACTION, random_state=
                       RANDOM_STATE):
194     if abs(train_fraction + val_fraction + test_fraction - 1.0) >
           1e-9:
195         raise ValueError("Las proporciones de split deben sumar 1.
               ")
196
197     games = np.unique(game_ids)
198     n = len(games)
199     train_end = int(n * train_fraction)
200     val_end = train_end + int(n * val_fraction)
201
202     split_of_game = np.empty(n, dtype=np.int8)
203     order = np.random.default_rng(random_state).permutation(n)
204     split_of_game[order[:train_end]] = 0
205     split_of_game[order[train_end:val_end]] = 1
206     split_of_game[order[val_end:]] = 2
207
208     position_split = split_of_game[np.searchsorted(games, game_ids
           )]
209     return tuple(np.flatnonzero(position_split == split) for split
            in range(3))
210
211
212 def run_etl(raw_path=RAW_DATA_PATH, processed_path=PROCESSED_PATH)
    :
213     if os.path.exists(processed_path):
214         print("Datos ya transformados. Saltando transformación.")
215         return load_data(processed_path)
216
217     print(f"Leyendo {raw_path}...")
218     data, counters = transform_data(read_games(raw_path))
219     save_data(data, counters, processed_path)
220     print("\nResumen del ETL:")
221     for name, value in counters.items():
222         print(f"  {name}: {value:,}")
223     print(f"Datos guardados en {processed_path}")
224     return data
225
```

```
226
227 if __name__ == "__main__":
228     data = run_etl()
229     train, val, test = split_data(data["game_id"])
230     print("\nPosiciones por conjunto (split por partida):")
231     print(f"Train:      {len(train):,}")
232     print(f"Validation: {len(val):,}")
233     print(f"Test:       {len(test):,}")
```

Análisis exploratorio.

```
import json
import os

import numpy as np

import plotUtil
from etlProcess import PROCESSED_PATH, load_data
from preprocessing import N_CLASSES

SUMMARY_PATH = "graficas/eda_resumen.json"

# Umbrales de fase decididos con el EDA.
OPENING_PLY = 20
ENDGAME_PIECES = 12
PHASE_NAMES = ("apertura", "medio juego", "final")

MAX_PLY_PLOT = 40
MIN_POSITIONS_PER_PLY = 100


def moved_piece(squares, move):
    return squares[np.arange(len(move)), move // 64]


def piece_count(squares):
    return (squares != 0).sum(axis=1)


def game_phase(ply, n_pieces, opening_ply=OPENING_PLY,
   endgame_pieces=ENDGAME_PIECES):
    phase = np.ones(len(ply), dtype=np.int8)
    phase[ply < opening_ply] = 0
    phase[n_pieces <= endgame_pieces] = 2
    return phase


def legal_counts(legal_offsets):
    return np.diff(legal_offsets)


def played_capture_value(move, legal_offsets, legal_moves,
   legal_capture):
    counts = legal_counts(legal_offsets)
    owner = np.repeat(np.arange(len(move)), counts)
    is_played = legal_moves == move[owner]
    values = np.zeros(len(move), dtype=np.int8)
    values[owner[is_played]] = legal_capture[is_played]
    return values


```

```
49 def max_capture_value(legal_offsets, legal_capture):
50     return np.maximum.reduceat(legal_capture, legal_offsets[:-1])
51
52
53 def position_keys(squares, castling, ep):
54     rows = np.ascontiguousarray(
55         np.column_stack([squares, castling, ep]).astype(np.int8)
56     )
57     as_bytes = rows.view(np.dtype((np.void, rows.shape[1])))[:, 0]
58     _, keys = np.unique(as_bytes, return_inverse=True)
59     return keys
60
61
62 def mode_moves(keys, move):
63     """Jugada más frecuente de cada posición (por llave); -1 si la
        llave no aparece."""
64     pairs = keys.astype(np.int64) * N_CLASSES + move
65     unique_pairs, counts = np.unique(pairs, return_counts=True)
66     pair_keys = unique_pairs // N_CLASSES
67     order = np.lexsort((-counts, pair_keys))
68     first = order[np.r_[True, np.diff(pair_keys[order]) != 0]]
69
70     modes = np.full(keys.max() + 1, -1, dtype=np.int32)
71     modes[pair_keys[first]] = unique_pairs[first] % N_CLASSES
72     return modes
73
74
75 def repeated_across_games(keys, game_id):
76     """True si la posición aparece en al menos dos partidas
        distintas."""
77     n_games = int(game_id.max()) + 1
78     key_game = np.unique(keys.astype(np.int64) * n_games + game_id
        )
79     games_per_key = np.bincount(key_game // n_games, minlength=
        keys.max() + 1)
80     return games_per_key[keys] > 1
81
82
83 def accuracy_ceiling(keys, move):
84     """True si la jugada es la modal de su posición (techo de top
        -1 al ver solo el tablero)."""
85     return mode_moves(keys, move)[keys] == move
86
87
88 def _share(values):
89     return round(float(values.mean()), 4) if len(values) else None
90
91
92 def _rate_by_ply(ply, values, mask=None, max_ply=MAX_PLY_PLOT,
    min_positions=MIN_POSITIONS_PER_PLY):
93     mask = np.ones(len(ply), bool) if mask is None else mask
```

```
    rates = np.full(max_ply + 1, np.nan)
    for p in range(max_ply + 1):
        at_ply = (ply == p) & mask
        if at_ply.sum() >= min_positions:
            rates[p] = values[at_ply].mean()
    return rates


def initial_graphs(data, summary_path=SUMMARY_PATH):
    plotUtil.ensure_graphics_directory()
    squares, move, ply, elo = data["squares"], data["move"], data["ply"], data["elo"]

    n_legal = legal_counts(data["legal_offsets"])
    piece = moved_piece(squares, move)
    n_pieces = piece_count(squares)
    phase = game_phase(ply, n_pieces)
    captured = played_capture_value(move, data["legal_offsets"], data["legal_moves"], data["legal_capture"])
    best_capture = max_capture_value(data["legal_offsets"], data["legal_capture"])
    keys = position_keys(squares, data["castling"], data["ep"])
    repeated = repeated_across_games(keys, data["game_id"])
    is_mode = accuracy_ceiling(keys, move)

    plotUtil.elo_hist(elo)
    plotUtil.ply_hist(ply)
    plotUtil.legal_moves_hist(n_legal)
    plotUtil.class_frequency(np.bincount(move, minlength=N_CLASSES))
    plotUtil.moved_piece_bar(piece)
    plotUtil.capture_bar(captured)
    plotUtil.phase_bar(phase, PHASE_NAMES)
    plotUtil.repeated_positions_by_ply(
        _rate_by_ply(ply, repeated),
        _rate_by_ply(ply, is_mode, repeated),
    )

    with_capture = best_capture > 0
    opening = phase == 0
    summary = {
        "posiciones": int(len(move)),
        "legales_media": round(float(n_legal.mean()), 2),
        "legales_min_max": [int(n_legal.min()), int(n_legal.max())],
        "azar_top1": round(float((1 / n_legal).mean()), 4),
        "ply_mayor_a_10": round(float((ply > 10).mean()), 4),
        "elo_p5_mediana_p95": [int(v) for v in np.percentile(elo, [5, 50, 95])],
        "elo_min_max": [int(elo.min()), int(elo.max())],
        "clases_que_aparecen": int((np.bincount(move, minlength=
```

```
            N_CLASSES) > 0).sum()),
        "pieza_movida": {name: round(float((piece == code).mean())
            , 4) for code, name in enumerate(
             ("peón", "caballo", "alfil", "torre", "dama", "rey"),
                start=1)},
        "jugada_es_captura": round(float((captured > 0).mean()),
            4),
        "hay_captura_disponible": round(float(with_capture.mean())
            , 4),
        "elige_captura_de_mayor_valor_si_hay": round(float((
            captured[with_capture] == best_capture[with_capture]).
            mean()), 4),
        "fases": {name: round(float((phase == code).mean()), 4)
            for code, name in enumerate(PHASE_NAMES)},
        "umbrales_fase": {"apertura_ply_menor_a": OPENING_PLY, "
            final_piezas_hasta": ENDGAME_PIECES},
        "repetidas_entre_partidas": _share(repeated),
        "repetidas_en_apertura": _share(repeated[opening]),
        "repetidas_fuera_de_apertura": _share(repeated[~opening]),
        "techo_top1_en_repetidas": _share(is_mode[repeated]),
        "techo_top1_posicion_inicial": _share(is_mode[ply == 0]),
    }
    os.makedirs(os.path.dirname(summary_path) or ".", exist_ok=
        True)
    with open(summary_path, "w", encoding="utf-8") as file:
        json.dump(summary, file, indent=2, ensure_ascii=False)
    return summary


if __name__ == "__main__":
    summary = initial_graphs(load_data(PROCESSED_PATH))
    print(json.dumps(summary, indent=2, ensure_ascii=False))
```

Gráficas.

```python
import os

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

DPI = 300
plt.rcParams.update({"font.size": 12})
GRAPHICS_DIR = "./graficas"
PIECE_NAMES = ("Peón", "Caballo", "Alfil", "Torre", "Dama", "Rey")


def ensure_graphics_directory():
    os.makedirs(GRAPHICS_DIR, exist_ok=True)


def _save(name):
    plt.tight_layout()
    plt.savefig(os.path.join(GRAPHICS_DIR, name), dpi=DPI)
    plt.close()


def elo_hist(elo):
    ensure_graphics_directory()
    plt.figure(figsize=(6, 4))
    plt.hist(elo, bins=80, color="steelblue")
    plt.axvline(np.median(elo), color="black", linestyle="--",
        label=f"Mediana = {np.median(elo):.0f}")
    plt.xlabel("Elo del jugador que mueve")
    plt.ylabel("Posiciones")
    plt.title("Distribución del Elo")
    plt.legend()
    _save("eda_elo.png")


def ply_hist(ply, cut=10):
    ensure_graphics_directory()
    plt.figure(figsize=(6, 4))
    plt.hist(ply, bins=np.arange(0, min(ply.max(), 200) + 2) -
        0.5, color="steelblue")
    plt.axvline(cut + 0.5, color="black", linestyle="--", label=f"
        ply = {cut}")
    plt.xlabel("Ply (medias jugadas desde el inicio)")
    plt.ylabel("Posiciones")
    plt.title("Ply de cada posición")
    plt.legend()
    _save("eda_ply.png")

```

```
48
49  def legal_moves_hist(n_legal):
50      ensure_graphics_directory()
51      fig, (left, right) = plt.subplots(1, 2, figsize=(12, 5.5))
52      left.hist(n_legal, bins=np.arange(1, n_legal.max() + 2) - 0.5,
            color="steelblue")
53      left.axvline(n_legal.mean(), color="black", linestyle="--",
            label=f"Media = {n_legal.mean():.1f}")
54      left.set_xlabel("Jugadas legales en la posición")
55      left.set_ylabel("Posiciones")
56      left.set_title("Número de jugadas legales")
57      left.legend()
58
59      edges = [1, 2, 5, 10, 20, 30, 40, 60, n_legal.max() + 1]
60      labels, hit_rate, share = [], [], []
61      for low, high in zip(edges[:-1], edges[1:]):
62          mask = (n_legal >= low) & (n_legal < high)
63          if not mask.any():
64              continue
65          labels.append(str(low) if high - 1 == low else f"{low}-{
                high - 1}")
66          hit_rate.append((1 / n_legal[mask]).mean() * 100)
67          share.append(mask.mean() * 100)
68      bars = right.bar(labels, hit_rate, color="darkorange")
69      for bar, pct in zip(bars, share):
70          right.text(bar.get_x() + bar.get_width() / 2, max(bar.
                get_height(), 6) + 1.5, f"{pct:.1f}%",
71                     ha="center", va="bottom", fontsize=9)
72      right.axhline((1 / n_legal).mean() * 100, color="black",
            linestyle="--",
73                    label=f"Promedio = {(1 / n_legal).mean() *
                        100:.1f}%")
74      right.set_xlabel("Jugadas legales")
75      right.set_ylabel("Acierto esperado al azar (%)")
76      right.set_title("Acierto del azar por jugadas legales")
77      right.set_ylim(0, 112)
78      right.tick_params(axis="x", rotation=45)
79      right.legend()
80      _save("eda_jugadas_legales.png")
81
82
83  def class_frequency(counts):
84      ensure_graphics_directory()
85      present = np.sort(counts[counts > 0])[::-1]
86      cumulative = np.cumsum(present) / present.sum()
87      plt.figure(figsize=(6.5, 4.5))
88      plt.plot(np.arange(1, len(present) + 1), cumulative * 100,
            color="steelblue")
89      for share in (0.5, 0.8, 0.95):
90          n_classes = np.searchsorted(cumulative, share) + 1
91          plt.plot([0, n_classes, n_classes], [share * 100, share *
```

```
            100, 0], color="gray", linestyle="--")
        plt.text(n_classes + 20, share * 100 - 6, f"{n_classes} jugadas: {share:.0%}")
    plt.xlim(0, len(present))
    plt.ylim(0, 102)
    plt.xlabel("Jugadas distintas, de la más a la menos común")
    plt.ylabel("% del total de jugadas")
    plt.title("Pocas jugadas concentran casi todo")
    _save("eda_frecuencia_clases.png")


def moved_piece_bar(piece):
    ensure_graphics_directory()
    shares = [(piece == code).mean() * 100 for code in range(1, 7)]
    plt.figure(figsize=(6, 4))
    bars = plt.bar(PIECE_NAMES, shares, color="steelblue")
    plt.bar_label(bars, fmt="%.1f%%")
    plt.ylim(0, max(shares) * 1.15)
    plt.ylabel("Porcentaje de las jugadas")
    plt.title("Pieza que se mueve")
    _save("eda_pieza_movida.png")


def capture_bar(captured):
    ensure_graphics_directory()
    values = [0, 1, 3, 5, 9]
    labels = ["Tranquila", "Peón (1)", "Caballo o\nalfil (3)", "Torre (5)", "Dama (9)"]
    shares = [(captured == v).mean() * 100 for v in values]
    plt.figure(figsize=(6.5, 4.5))
    bars = plt.bar(labels, shares, color=["gray"] + ["firebrick"] * 4)
    plt.bar_label(bars, fmt="%.1f%%")
    plt.ylim(0, max(shares) * 1.15)
    plt.ylabel("Porcentaje de las jugadas")
    plt.title("Valor de la pieza capturada")
    _save("eda_capturas.png")


def phase_bar(phase, phase_names):
    ensure_graphics_directory()
    shares = [(phase == code).mean() * 100 for code in range(len(phase_names))]
    plt.figure(figsize=(5.5, 4))
    bars = plt.bar([name.capitalize() for name in phase_names], shares, color="steelblue")
    plt.bar_label(bars, fmt="%.1f%%")
    plt.ylim(0, max(shares) * 1.15)
    plt.ylabel("Porcentaje de las posiciones")
    plt.title("Fase de la partida")
```

```
136     _save("eda_fases.png")
137
138
139 def repeated_positions_by_ply(repeated_rate, ceiling_rate):
140     ensure_graphics_directory()
141     plies = np.arange(len(repeated_rate))
142     plt.figure(figsize=(8, 4.5))
143     plt.plot(plies, repeated_rate * 100, marker=".", label="
            Posiciones que aparecen en 2 o más partidas")
144     plt.plot(plies, ceiling_rate * 100, marker=".", label="Techo
            de top-1 en esas posiciones")
145     plt.xlabel("Ply")
146     plt.ylabel("Porcentaje")
147     plt.ylim(0, 100)
148     plt.title("Posiciones que se repiten entre partidas, por ply")
149     plt.legend()
150     _save("eda_posiciones_repetidas.png")
```

Referencias

[1] Muhammad Faiz Arsalan y Haryono Soeparno. "Deep Learning Approaches to Predicting the Optimal Chess Moves from Board Positions". En: *International Journal of Engineering Trends and Technology* 72.4 (2024), págs. 43-50. DOI: 10.14445/22315381/IJETT-V72I4P105.

[2] Timo Bertram, Johannes Fürnkranz y Martin Müller. "Neural Network-Based Information Set Weighting for Playing Reconnaissance Blind Chess". En: *IEEE Transactions on Games* 16 (2024), págs. 960-970. DOI: 10.1109/TG.2024.3425803.

[3] Niklas Fiekas. *python-chess: a chess library for Python*. https://python-chess.readthedocs.io/. Versión 1.11.2. [Consultado 15-10-2026].

[4] Lichess. *Lichess Open Database*. https://database.lichess.org/. Licencia CC0. [Consultado 15-10-2026].

[5] Reid McIlroy-Young et al. "Aligning Superhuman AI with Human Behavior: Chess as a Model System". En: *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. KDD '20. ACM, 2020, págs. 1677-1687. DOI: 10.1145/3394486.3403219.

[6] Heigon Soldera y Helio Pedrini. "Chess Board Digitization and Human Move Prediction Using Machine Learning". En: *IEEE Access* 14 (2026), págs. 81589-81615. DOI: 10.1109/ACCESS.2026.3697295.